

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

AI-Based Phishing & Social Engineering Detection System

ASSIGNMENT A2: SOFTWARE REQUIREMENTS SPECIFICATION

TEAM MEMBERS & CONTRIBUTIONS

PRN	Team Member Name
202301040020	Yash Mehenge
202301040045	Omkar Gavali
202301040047	Abhishek Verma
202301040049	Laxman Adhikari
202301040267	Sneha Nannaware

Document Control

Version: 1.0 (SRS Draft for Assignment 2)

Date: August 31, 2026

Status: Completed Specification based on Assignment-01 Component Identification.

Target Environment: Enterprise IT and Web Client Architectures.

TABLE OF CONTENTS

1. Introduction & Objectives	Page 3
1.1 Purpose of the Document	Page 3
1.2 Scope of the System	Page 3
1.3 Objectives & Relevancy	Page 3
2. Overall Description & Architecture	Page 4
2.1 Product Perspective & Layered Model	Page 4
2.2 Operating Environment & Context	Page 4
2.3 User Classes & Characteristics	Page 5
3. Functional Requirements	Page 5
3.1 FR-1: Content Ingestion & Preprocessing	Page 5
3.2 FR-2: Feature Extraction Pipeline	Page 5
3.3 FR-3: AI Classifier Inference	Page 5
3.4 FR-4: Real-Time Warning Interface	Page 6
3.5 FR-5: Analyst Dashboard & Incidents Database	Page 6
4. Non-Functional Requirements	Page 6
4.1 Performance (Latency & Throughput)	Page 6
4.2 Security, Privacy & Integrity	Page 7
4.3 Reliability & Resilience	Page 7
5. User & Actor Requirements	Page 7
6. System Constraints	Page 7
7. Assumptions and Dependencies	Page 8
8. External Interface Requirements	Page 8
9. Requirements Traceability Matrix (RTM)	Page 9
10. Conclusion & References	Page 10

1. INTRODUCTION & OBJECTIVES

Phishing and social engineering attacks remain one of the most common and damaging entry points for cyberattacks worldwide. Adversaries craft deceptive emails, SMS messages, chat messages, and fake web pages that impersonate trusted individuals or organizations to trick users into revealing credentials, financial information, or installing malware. Traditional rule-based spam and phishing filters (such as keyword blacklists, static URL blocklists, and sender reputation lists) struggle to keep up with constantly evolving attack patterns. These rules fail against AI-generated phishing content, homograph domains, and highly personalized spear-phishing messages. This Software Requirements Specification (SRS) outlines the requirements for designing and developing an AI-based Phishing and Social Engineering Detection System that intercepts and analyzes text and link content in near-real-time using advanced Natural Language Processing (NLP) and machine learning classification algorithms.

1.1 Purpose of the Document

This SRS document serves as the formal foundation for the design, development, testing, and deployment of the AI-Based Phishing and Social Engineering Detection System. It outlines all functional and non-functional requirements, system capabilities, actors, operational constraints, and external interface descriptions. It is designed to ensure alignment between the system architecture, the machine learning development team, enterprise IT security stakeholders, and academic assessors of this project.

1.2 Scope of the System

The proposed system functions as a data-driven, AI/ML-based intelligent detection and decision-support system. The initial prototype focuses on the following core scopes:

(a) Ingestion and Preprocessing: Capturing text and headers from personal/corporate email platforms, chat platforms, and rendered web page HTML content.

(b) Automated Feature Extraction: Analyzing linguistic cues indicating psychological manipulation (such as urgency, authority, fear-based patterns), examining domain/URL characteristics, and processing sender metadata.

(c) Intelligent AI Classification Pipeline: Scoring ingested data via trained Machine Learning/NLP models to generate a decimal suspicion score and categorical hazard label (e.g., Safe, Suspicious, High-Risk Phishing).

(d) User Warning & Alerting Interface: Presenting immediate in-context alerts to individual end users via web/browser extensions and email add-ins, and routing aggregated records to an incident dashboard for cybersecurity analysts.

Out of Scope for initial prototype: Network/mail server packet interception, automated legal or hosting provider takedown actions, and massive multi-tenant ISP-level deployment.

1.3 Objectives & Domain Relevancy

With the rapid rise of generative AI tools, attackers can easily produce grammatically flawless, highly convincing spear-phishing emails at scale. Consequently, rule-based security systems suffer from high false-negative and false-positive rates. The core objective of this system is to deploy learning-based NLP algorithms that detect semantic exploitation, and to construct a decoupled, horizontal architecture that updates models dynamically without disrupting client-side integrations. This directly addresses the cybersecurity industry's urgent need for content-aware detection, making it an excellent applied Artificial Intelligence project.

2. OVERALL DESCRIPTION & ARCHITECTURE

The system is a hybrid content-analysis, client-server, and near-real-time monitoring system designed to score incoming communications at the point of consumption. It provides immediate warnings to users reading the messages, while transmitting detection telemetry to a security analytics data layer for centralized compliance monitoring.

2.1 Product Perspective & Layered Model

The system uses a highly modular **Layered Client-Server Architecture**. This allows developers to optimize computationally expensive NLP inference separately from lightweight user interfaces. The five layers are structured as follows:

Layer	Component	Justification
Presentation Layer	Browser extension, email add-in, web dashboard	Provides lightweight interfaces at point of consumption for in-context alerts.
Application / API	REST API gateway, orchestrator, authentication	Decouples client software from ML models, allowing hot-swapping and API reusability.
AI / ML Processing	NLP feature extraction, NLP classifiers, URL checker	Centralizes model execution; features are extracted and run on GPUs for low latency.
Data Layer	Model store, incident DB, threat-intel feeds	Persists logs for compliance, maintains model registry, and aggregates threat lists.
Deployment	Cloud-hosted microservices, containerized ML	Ensures high scalability and dynamic horizontal scaling under heavy load.

Note: A detailed visualization of the component boundaries and interfaces across these five layers is provided in Figure 5 of the accompanying UML Diagrams document.

2.2 Operating Environment & Context

The AI Phishing Detection System is designed to operate within modern personal and corporate productivity tools. The client-side scripts execute as a browser extension (compatible with Chrome, Firefox, and Edge) and an email add-in (compatible with Microsoft Outlook and Google Workspace API integrations). The backend runs on Unix-based containerized cloud infrastructure (Docker & Kubernetes) supporting high-performance Python classification services.

2.3 User Classes & Characteristics

The system has two primary human user classes, each with distinct technical expertise and workflows:

- 1. Individual End User:** Typically non-technical. These users encounter emails, chats, or URLs in their daily life. They require zero-configuration, simple, clear, and highly visible warnings (e.g., color-coded banners) directly inside their communication client without navigating to external applications.
- 2. Security Analyst / Enterprise IT Team:** High technical expertise. They monitor the aggregated dashboard, investigate reported or flagged malicious incidents, manage threat intelligence rules, configure alert parameters, and retrain ML classification pipelines.

3. FUNCTIONAL REQUIREMENTS

Functional requirements detail the specific operations, calculations, inputs, and outputs that the system must execute. These requirements are directly mapped to the scope components (a), (b), (c), and (d).

ID	Functional Requirement Description	Priority	Core Outputs
FR-1.1	Email Client Ingestion: Parse email headers, sender address, routing path, HTML body, and plain-text body.	High	Structured Content Object
FR-1.2	Chat Platform Ingestion: Retrieve incoming chat messages via browser extension injection or API webhook.	High	Parsed Message Payload
FR-1.3	Web Content Ingestion: Parse page title, DOM links, anchor tags, and raw body text from active browser window.	Medium	Rendered Text & URL Map
FR-2.1	Linguistic NLP Feature Extraction: Analyze semantic cues indicating urgency, fear, authority, and financial keywords.	High	Linguistic Feature Vector
FR-2.2	URL Analysis Engine: Extract domain age, SSL status, homograph patterns, and cross-reference with blocklists.	High	URL Health Score
FR-2.3	Sender Metadata Verification: Validate DKIM, SPF, and DMARC alignments, and look up IP reputations.	Medium	Sender Authenticity Vector
FR-3.1	ML Classification Inference: Feed combined feature vector to the containerized NLP models on GPU.	High	Phishing Classification Score
FR-3.2	Score Output & Category Assignment: Return suspicion value [0.0 - 1.0] and hazard level (Safe, Warning, Phishing).	High	Validated Classification Score
FR-4.1	Client UI Alerting: Inject color-coded warning banners (Red for danger, Yellow for suspicious) directly into UI.	High	Active User Alert Banner
FR-5.1	Central Incident Logging: Record classification metadata, timestamps, flagged content (anonymized), and score to DB.	High	PostgreSQL Log Record
FR-5.2	Analyst Dashboard View: Present a centralized web-based interface visualizing incident trends, logs, and stats.	Medium	Analytics Graphs & Audit Logs
FR-6.1	Model Management & Retraining: Allow analysts to trigger manual retraining or hot-swap production ML models.	Low	Updated Classifier weights

4. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the operational criteria, constraints, and quality gates that the system must satisfy to ensure usability, security, and enterprise viability in production environments.

4.1 Performance & Latency Requirements (NFR-1)

NFR-1.1: Inference Speed: The backend AI classification model must output a suspicion score and label in less than **200 milliseconds** from the moment features are received at the REST API Gateway.

NFR-1.2: End-to-End Latency: The total round-trip time (ingestion -> feature extraction -> model scoring -> UI rendering) must not exceed **500 milliseconds**, preventing any noticeable lag or disruption in the user's reading workflow.

NFR-1.3: Concurrency & Scalability: The Application and AI layers must support up to **1,000 concurrent classification requests** per second, using container orchestration (Kubernetes) to dynamically scale inference microservices horizontally during high-volume periods.

4.2 Security, Privacy & Integrity (NFR-2)

NFR-2.1: Data Transit Encryption: All communications between the client integrations (browser extensions/mail add-ins) and the API Gateway must be encrypted in transit using **TLS 1.3**.

NFR-2.2: PII Anonymization & Privacy: Personal user information, email credentials, and message content must be anonymized before logging to the Data Layer. The system must operate under GDPR and data privacy compliance standards. Raw body text must never be persisted in plain text within the incident database.

NFR-2.3: API Authentication: Access to backend API endpoints by client devices must be authenticated using JSON Web Tokens (JWT) with short expiration windows, preventing unauthorized spoofing of classification queries.

4.3 Reliability & Resilience (NFR-3)

NFR-3.1: Service Availability: The API gateway and AI processing layer must achieve **99.9% uptime** (excluding planned maintenance windows).

NFR-3.2: Graceful Failure & Local Fallbacks: If the connection to the cloud backend is lost or the AI inference layer becomes unreachable, the client extension must gracefully fall back to a lightweight, local rule-based blacklist (caching popular malicious URLs/senders locally) so the user remains partially protected, rather than failing silently or blocking legitimate work.

5. USER & ACTOR REQUIREMENTS

The system enforces detailed, actor-specific requirements to align with their distinct operational workflows:

5.1 Individual End User Requirements:

- The system must require zero cybersecurity knowledge from the user; phishing indicators must be explained via tooltips.
- A settings toggle must allow users to white-list specific trusted domains locally to reduce false-positive warnings.
- The warning banner must allow the user to 'disregard and proceed' or 'mark as safe' to update local caching policies.

5.2 Security Analyst / IT Team Requirements:

- The security team must be able to search and filter logs by date range, suspicion score, content type, and targeted employee.
- Analysts must receive instantaneous alerts for 'High-Risk Phishing' classifications to trigger active internal security responses.
- The administration interface must provide detailed ROC curves, precision/recall metrics, and confusion matrices for the active NLP model.

6. SYSTEM CONSTRAINTS

6.1 Computational Limits on Clients: Client integrations (extensions and add-ins) must consume minimal memory (under 50 MB RAM) and negligible CPU on host devices to ensure user device performance is unaffected.

6.2 Strict Air-Gapped Network Isolation (Security Policy): Consistent with secure enterprise deployments, the core AI Processing microservices and the Data Layer must run inside private, isolated networks with no public egress connections. Telemetry data and threat intelligence lists must be synced through controlled secure proxy servers.

6.3 Model Size & Deployment Boundaries: Containerized classification models must be optimized to fit standard cloud instance configurations, using quantization (e.g., INT8) to reduce model sizes under 1.5 GB for quick auto-scaling.

7. ASSUMPTIONS & DEPENDENCIES

7.1 Assumptions:

- It is assumed that curated, high-quality, labeled phishing and legitimate datasets are available for training and retraining the NLP models.
- It is assumed that end users have a stable, low-latency internet connection to reach the REST API Gateway.
- It is assumed that enterprise mail systems allow browser extension side-loading or email add-in deployment permissions.

7.2 Dependencies:

- The system depends on third-party threat intelligence APIs and open-source feeds for domain registration data and IP blacklists.
- The system depends on the availability of container orchestrators (e.g., Kubernetes) for scaling deep learning classification models.

8. EXTERNAL INTERFACE REQUIREMENTS

The system interacts with several external entities, requiring precise software, hardware, and communication interfaces.

8.1 User Interfaces (UI)

Browser Extension UI: Built using HTML, CSS, and vanilla JavaScript. It injects dynamic warning banners into active webpages. On click, it opens a popup indicating specific risk factors (e.g., 'Warning: Domain was registered 2 days ago').

Email Add-in UI: A native HTML pane rendered in Outlook / Google Workspace, highlighting sender verification status (SPF/DKIM) and placing a red warning bar on suspicious messages.

Analyst Dashboard: A responsive web portal built with React and Tailwind CSS, presenting graphical trend charts, log tables, and admin configurations.

8.2 Software & Hardware Interfaces

Software Interfaces: The REST API uses JSON formatted requests. The AI layer integrates with TensorFlow/PyTorch runtimes and MLflow. External threat interfaces query security database vendors and WHOIS databases.

Hardware Interfaces: Host devices require standard internet access. Backend API and ML services require GPU accelerators (e.g., NVIDIA T4) to meet the 200ms latency constraint.

8.3 Communication Interfaces

All internal microservice API communications must use **gRPC** for high-performance serialized messaging. All client-to-API communications use RESTful HTTP/2 over TLS 1.3. Telemetry reports use persistent secure WebSockets for live logging to the security analyst console.

9. REQUIREMENTS TRACEABILITY MATRIX (RTM)

This matrix traces each functional requirement back to its corresponding system actor, use case, and priority, ensuring complete coverage and verification.

Req ID	Primary Functional Objective	Mapped Actor	Mapped Use Case	Priority
FR-1.1	Email Header & Body Ingestion	Mail Client	UC1: Ingest Content	High
FR-1.2	Chat Message Content Capture	Browser Ext	UC1: Ingest Content	High
FR-1.3	DOM Links & Web Text Ingestion	Browser Ext	UC1: Ingest Content	Medium
FR-2.1	Linguistic NLP Features (Urgency)	AI Engine	UC2: Extract Features	High
FR-2.2	URL Homograph & Reputation Checks	AI Engine	UC2: Extract Features	High
FR-2.3	Sender Metadata Alignment	AI Engine	UC2: Extract Features	Medium
FR-3.1	ML Inference pipeline	ML Classifier	UC3: Perform AI Classification	High
FR-3.2	Suspicion Score & Label Output	ML Classifier	UC3: Perform AI Classification	High
FR-4.1	Dynamic Red/Yellow Warning Banner	End User	UC4: View In-Context Alerts	High

Req ID	Primary Functional Objective	Mapped Actor	Mapped Use Case	Priority
FR-5.1	Incident Database Log Storage	API Gateway	UC5: View Dashboard & Logs	High
FR-5.2	Analyst Dashboard Visual Logs	Analyst	UC6: Investigate Incidents	Medium
FR-6.1	Dynamic Model Retraining	Analyst	UC7: Manage Model Store	Low

10. CONCLUSION & REFERENCES

10.1 Conclusion:

The Software Requirements Specification (SRS) for the AI-Based Phishing and Social Engineering Detection System outlines a scalable, secure, and near-real-time solution. By separating interface code from heavy Natural Language Processing classifiers via REST APIs and cloud microservices, the system guarantees low latency and hot-swappable AI logic. This system directly addresses a critical vector in cybersecurity. This document ensures clear development tracks, consistent interfaces, and a solid foundation for the subsequent UML architectural design models.

10.2 Academic References:

1. Yash Mehenge, Omkar Gavali, Abhishek Verma, Laxman Adhikari, Sneha Nannaware. *AI-Based Phishing & Social Engineering Detection System - Assignment A1: Component Problem Statement Identification*, Department of Artificial Intelligence & Cybersecurity, 2026.
2. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
3. Standard Cybersecurity Threat Reports on Generative AI Phishing Vectors (2024-2026).